Lab 1. PyTorch and ANNs

This lab is a warm up to get you used to the PyTorch programming environment used in the course, and also to help you review and renew your knowledge of Python and relevant Python libraries. The lab must be done individually. Please recall that the University of Toronto plagarism rules apply.

By the end of this lab, you should be able to:

1. Be able to perform basic PyTorch tensor operations.
2. Be able to load data into PyTorch
3. Be able to configure an Artificial Neural Network (ANN) using PyTorch
4. Be able to train ANNs using PyTorch
5. Be able to evaluate different ANN configuations

You will need to use numpy and PyTorch documentations for this assignment:

- https://docs.scipy.org/doc/numpy/reference/
- https://pytorch.org/docs/stable/torch.html

You can also reference Python API documentations freely.

What to submit

Submit a PDF file containing all your code, outputs, and write-up from parts 1-5. You can produce a PDF of your Google Colab file by going to `File -> Print` and then save as PDF. The Colab instructions has more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Please use Google Colab to complete this assignment. If you want to use Jupyter Notebook, please complete the assignment and upload your Jupyter Notebook file to Google Colab for submission.

Adjust the scaling to ensure that the text is not cutoff at the margins.

Colab Link

Submit make sure to include a link to your colab file here

Colab Link: https://colab.research.google.com/drive/1_R0iG1ET_OSBAuq2yZ6l0-X_7xWtHdiV#scrollTo=_TlLBrWBIGt0

Part 1. Python Basics [3 pt]

The purpose of this section is to get you used to the basics of Python, including working with functions, numbers, lists, and strings.

Note that we **will** be checking your code for clarity and efficiency.

If you have trouble with this part of the assignment, please review http://cs231n.github.io/python-numpy-tutorial/

Part (a) -- 1pt

Write a function `sum_of_cubes` that computes the sum of cubes up to `n`. If the input to `sum_of_cubes` invalid (e.g. negative or non-integer `n`), the function should print out `"Invalid input"` and return `-1`.

```
def sum_of_cubes(n):
    """Return the sum (1^3 + 2^3 + 3^3 + ... + n^3)

    Precondition: n > 0, type(n) == int

    >>> sum_of_cubes(3)
    36
    >>> sum_of_cubes(1)
    1
    """
    sum = 0
    if n < 1 or type(n) != int:
        print("Invalid input")
        return -1
    else:
        for i in range (n+1):
            sum = sum + i*i*i

    return sum

print (sum_of_cubes(0.5))
print (sum_of_cubes(3))
```

```
Invalid input
-1
36
```

Part (b) -- 1pt

Write a function `word_lengths` that takes a sentence (string), computes the length of each word in that sentence, and returns the length of each word in a list. You can assume that words are always separated by a space character `" "`.

Hint: recall the `str.split` function in Python. If you arenot sure how this function works, try typing `help(str.split)` into a Python shell, or check out https://docs.python.org/3.6/library/stdtypes.html#str.split

```
help(str.split)
```

```
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1) unbound builtins.str method
    Return a list of the substrings in the string, using sep as the separa

      sep
        The separator used to split the string.

        When set to None (the default value), will split on any whitespac
        character (including \n \r \t \f and spaces) and will discard
        empty strings from the result.
      maxsplit
        Maximum number of splits.
        -1 (the default value) means no limit.

    Splitting starts at the front of the string and works to the end.

    Note, str.split() is mainly useful for data that has been intentional
    delimited.  With natural text that includes punctuation, consider usi
    the regular expression module.
```

```
def word_lengths(sentence):
    """Return a list containing the length of each word in
    sentence.

    >>> word_lengths("welcome to APS360!")
    [7, 2, 7]
    >>> word_lengths("machine learning is so cool")
    [7, 8, 2, 2, 4]
    """
    list_of_words = str.split(sentence)
    lengths_of_words = []
    for word in list_of_words:
        lengths_of_words.append(len(word))
```

```
    return lengths_of_words
```

```
word_lengths("my name is alex")
```

```
[2, 4, 2, 4]
```

Part (c) -- 1pt

Write a function `all_same_length` that takes a sentence (string), and checks whether every word in the string is the same length. You should call the function `word_lengths` in the body of this new function.

```
def all_same_length(sentence):
    """Return True if every word in sentence has the same
    length, and False otherwise.

    >>> all_same_length("all same length")
    False
    >>> word_lengths("hello world")
    True
    """
    lengths_list = word_lengths(sentence)
    same = True
    for length in lengths_list:
        if length != lengths_list[0]:
            same = False
            break

    return same
```

```
all_same_length("what is my name")
```

```
False
```

```
all_same_length("hello world")
```

```
True
```

Part 2. NumPy Exercises [5 pt]

In this part of the assignment, you'll be manipulating arrays usign NumPy. Normally, we use the shorter name `np` to represent the package `numpy`.

```
import numpy as np
```

Part (a) -- 1pt

The below variables `matrix` and `vector` are numpy arrays. Explain what you think `<NumpyArray>.size` and `<NumpyArray>.shape` represent.

Part(a) Answer:

1. `<NumpyArray>.size` represents the total number of entries/ elements insides the numpy arrays
2. `<NumpyArray>.shape` represents the size along each dimension of the numPy array. For a matrix (2D array), it is (number_of_rows, number_of_columns). For a vector (1D array), it is (number_of_elements,)

```
matrix = np.array([[1., 2., 3., 0.5],
                   [4., 5., 0., 0.],
                   [-1., -2., 1., 1.]])
vector = np.array([2., 0., 1., -2.])
```

```
matrix.size
```

```
12
```

```
matrix.shape
```

```
(3, 4)
```

```
vector.size
```

```
4
```

```
vector.shape
```

```
(4,)
```

Part (b) -- 1pt

Perform matrix multiplication `output = matrix x vector` by using for loops to iterate through the columns and rows. Do not use any builtin NumPy functions. Cast your output into a NumPy array, if it isn't one already.

Hint: be mindful of the dimension of output

```
output=[]
for row in matrix:
    sum = 0
    for i in range(len(row)):
        sum += row[i] * vector[i]
    output.append(sum)
```

```
output= np.array(output)
print(output)
```

```
[ 4.  8. -3.]
```

Part (c) -- 1pt

Perform matrix multiplication `output2 = matrix x vector` by using the function `numpy.dot`.

We will never actually write code as in part(c), not only because `numpy.dot` is more concise and easier to read/write, but also performance-wise `numpy.dot` is much faster (it is written in C and highly optimized). In general, we will avoid for loops in our code.

```
output2 = np.dot(matrix, vector)
```

```
output2
```

```
array([ 4.,  8., -3.])
```

Part (d) -- 1pt

As a way to test for consistency, show that the two outputs match.

```
match = True
for i in range(len(output)):
    if output[i] != output2[i]:
        match = False
        break

print (match)
```

```
True
```

Part (e) -- 1pt

Show that using `np.dot` is faster than using your code from part (c).

You may find the below code snippit helpful:

```
import time

# record the time before running code
start_time = time.time()

# place code to run here
output=[]
for row in matrix:
    sum = 0
    for i in range(len(row)):
        sum += row[i] * vector[i]

    output.append(sum)

print(output)

# record the time after the code is run
end_time = time.time()

# compute the difference
diff_forloop = end_time - start_time
print("For loop time:", diff_forloop)
```

```
[np.float64(4.0), np.float64(8.0), np.float64(-3.0)]
For loop time: 0.0015001296997070312
```

```
import time

# record the time before running code
start_time = time.time()

# place code to run here
output2 = np.dot(matrix, vector)
print(output2)

# record the time after the code is run
end_time = time.time()

# compute the difference
diff_npdot = end_time - start_time
print("npdot time:", diff_npdot)
```

```
[ 4.  8. -3.]
npdot time: 0.002053976058959961
```

Part 3. Images [6 pt]

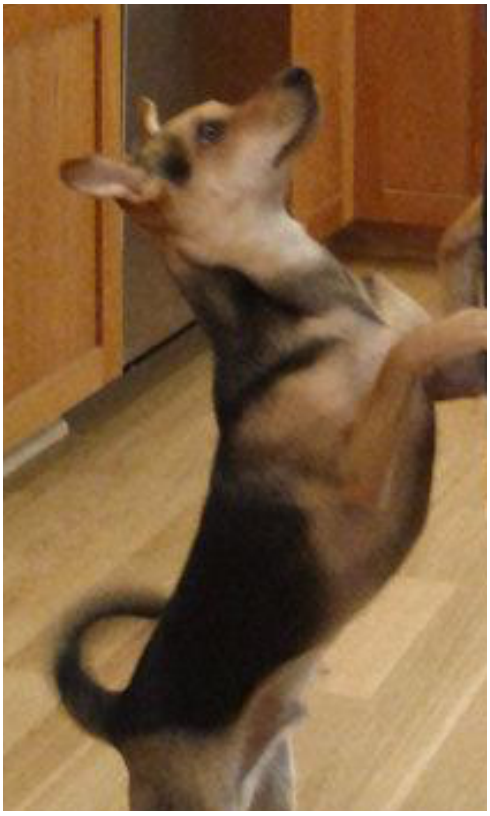
A picture or image can be represented as a NumPy array of “pixels”, with dimensions H × W × C, where H is the height of the image, W is the width of the image, and C is the number of colour channels. Typically we will use an image with channels that give the the Red, Green, and Blue “level” of each pixel, which is referred to with the short form RGB.

You will write Python code to load an image, and perform several array manipulations to the image and visualize their effects.

```
import matplotlib.pyplot as plt
```

Part (a) -- 1 pt

This is a photograph of a dog whose name is Mochi.



Load the image from its url (https://drive.google.com/uc?export=view&id=1oaLVR2hr1_qzpKQ47i9rVUIklwbDcews) into the variable `img` using the `plt.imread` function.

Hint: You can enter the URL directly into the `plt.imread` function as a Python string.

```
import urllib.request

url = "https://drive.google.com/uc?export=view&id=1oaLVR2hr1_qzpKQ47i9rV
urllib.request.urlretrieve(url, "dog.png")

img = plt.imread("dog.png")
```

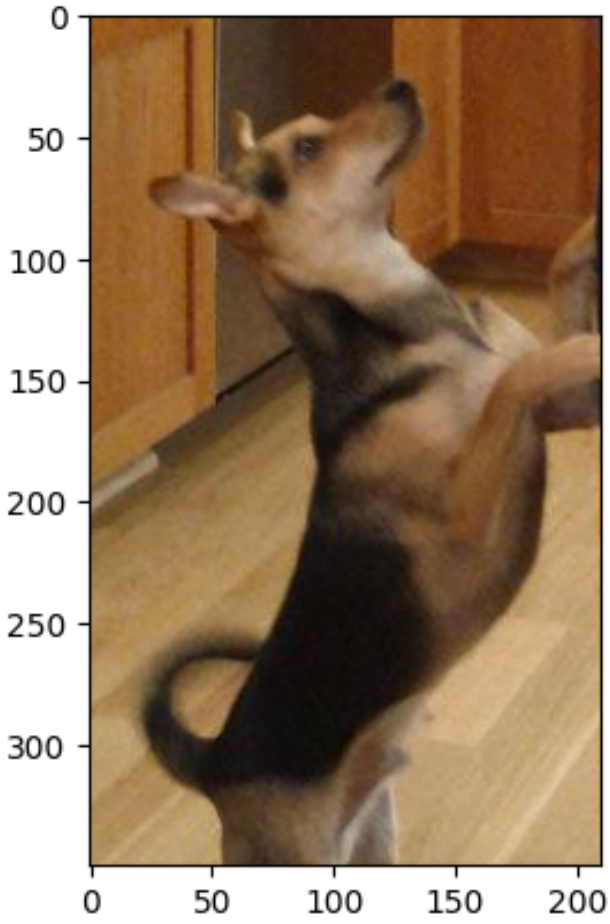
Part (b) -- 1pt

Use the function `plt.imshow` to visualize `img`.

This function will also show the coordinate system used to identify pixels. The origin is at the top left corner, and the first dimension indicates the Y (row) direction, and the second dimension indicates the X (column) dimension.

```
plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x7a816fc50aa0>
```

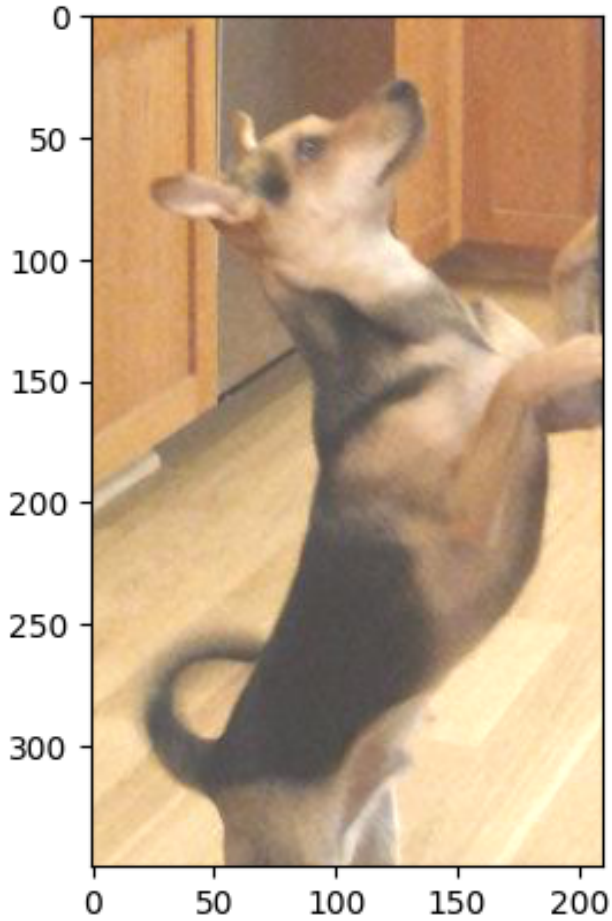


Part (c) -- 2pt

Modify the image by adding a constant value of 0.25 to each pixel in the `img` and store the result in the variable `img_add`. Note that, since the range for the pixels needs to be between [0, 1], you will also need to clip img_add to be in the range [0, 1] using `numpy.clip`. Clipping sets any value that is outside of the desired range to the closest endpoint. Display the image using `plt.imshow`.

```
img_add =np.clip(img + 0.25, 0, 1)
plt.imshow(img_add)
```

```
<matplotlib.image.AxesImage at 0x7a816fb184a0>
```



Part (d) -- 2pt

Crop the **original** image (`img` variable) to a 130 x 150 image including Mochi's face. Discard the alpha colour channel (i.e. resulting `img_cropped` should **only have RGB channels**)

Display the image.

```
img_cropped = img[0:130, 0:150, :3]
plt.imshow(img_cropped)
```

```
<matplotlib.image.AxesImage at 0x7a8177558650>
```

Part 4. Basics of PyTorch [6 pt]

PyTorch is a Python-based neural networks package. Along with tensorflow, PyTorch is currently one of the most popular machine learning libraries.

PyTorch, at its core, is similar to Numpy in a sense that they both try to make it easier to write codes for scientific computing achieve improved performance over vanilla Python by leveraging highly optimized C back-end. However, compare to Numpy, PyTorch offers much better GPU support and provides many high-level features for machine learning. Technically, Numpy can be used to perform almost every thing PyTorch does. However, Numpy would be a lot slower than PyTorch, especially with CUDA GPU, and it would take more effort to write machine learning related code compared to using PyTorch.

```
import torch
```

Part (a) -- 1 pt

Use the function `torch.from_numpy` to convert the numpy array `img_cropped` into a PyTorch tensor. Save the result in a variable called `img_torch`.

```
img_torch = torch.from_numpy(img_cropped)
```

Part (b) -- 1pt

Use the method `<Tensor>.shape` to find the shape (dimension and size) of `img_torch`.

```
img_torch.shape
```

```
torch.Size([130, 150, 3])
```

Part (c) -- 1pt

How many floating-point numbers are stored in the tensor `img_torch`?

```
FloatingPoint_numbers = img_torch.shape[0] * img_torch.shape[1] * img_to
print(FloatingPoint_numbers)
```

```
58500
```

Part (d) -- 1 pt

What does the code `img_torch.transpose(0,2)` do? What does the expression return? Is the original variable `img_torch` updated? Explain.

Part (d) Answer:

1. It swaps dimension 0 and dimension 2
2. The expression just returns [3, 150, 130]
3. img_torch is not updated

```
# It swaps dimension 0 and dimension 2
print(img_torch.transpose(0,2).shape)

img_torch.shape
```

```
torch.Size([3, 150, 130])
torch.Size([130, 150, 3])
```

Part (e) -- 1 pt

What does the code `img_torch.unsqueeze(0)` do? What does the expression return? Is the original variable `img_torch` updated? Explain.

Part(e) Answer:

1. It adds a dimension at dimension 0 with size 1
2. The expression `img_torch.unsqueeze(0)` returns torch.Size([1, 130, 150, 3])
3. img_torch is not being updated

```
print(img_torch.unsqueeze(0).shape)
print(img_torch.shape)
```

```
torch.Size([1, 130, 150, 3])
torch.Size([130, 150, 3])
```

Part (f) -- 1 pt

Find the maximum value of `img_torch` along each colour channel? Your output should be a one-dimensional PyTorch tensor with exactly three values.

Hint: lookup the function `torch.max`.

```
max_per_channel = torch.max(torch.max(img_torch, dim=0).values, dim=0).v
max_per_channel
```

```
tensor([0.8941, 0.7882, 0.6745])
```

Part 5. Training an ANN [10 pt]

The sample code provided below is a 2-layer ANN trained on the MNIST dataset to identify digits less than 3 or greater than and equal to 3. Modify the code by changing any of the following and observe how the accuracy and error are affected:

- number of training iterations
- number of hidden units

- numbers of layers
- types of activation functions
- learning rate

Please select at least three different options from the list above. For each option, please select two to three different parameters and provide a table.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
import matplotlib.pyplot as plt # for plotting
import torch.optim as optim

torch.manual_seed(1) # set the random seed

# define a 2-layer artificial neural network
class Pigeon(nn.Module):
    def __init__(self):
        super(Pigeon, self).__init__()
        self.layer1 = nn.Linear(28 * 28, 30)
        self.layer2 = nn.Linear(30, 1)
    def forward(self, img):
        flattened = img.view(-1, 28 * 28)
        activation1 = self.layer1(flattened)
        activation1 = F.relu(activation1)
        activation2 = self.layer2(activation1)
        return activation2

pigeon = Pigeon()

# load the data
mnist_data = datasets.MNIST('data', train=True, download=True)
mnist_data = list(mnist_data)
mnist_train = mnist_data[:1000]
mnist_val   = mnist_data[1000:2000]
img_to_tensor = transforms.ToTensor()


# simplified training code to train `pigeon` on the "small digit recogni
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(pigeon.parameters(), lr=0.005, momentum=0.9)

for (image, label) in mnist_train:
    # actual ground truth: is the digit less than 3?
    actual = torch.tensor(label < 3).reshape([1,1]).type(torch.FloatTens
    # pigeon prediction
    out = pigeon(img_to_tensor(image)) # step 1-2
    # update the parameters based on the loss
    loss = criterion(out, actual)      # step 3
```

```
    loss.backward()                     # step 4 (compute the updates for
    optimizer.step()                    # step 4 (make the updates for ea
    optimizer.zero_grad()               # a clean up step for PyTorch

# computing the error and accuracy on the training set
error = 0
for (image, label) in mnist_train:
    prob = torch.sigmoid(pigeon(img_to_tensor(image)))
    if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
        error += 1
print("Training Error Rate:", error/len(mnist_train))
print("Training Accuracy:", 1 - error/len(mnist_train))


# computing the error and accuracy on a test set
error = 0
for (image, label) in mnist_val:
    prob = torch.sigmoid(pigeon(img_to_tensor(image)))
    if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
        error += 1
print("Test Error Rate:", error/len(mnist_val))
print("Test Accuracy:", 1 - error/len(mnist_val))
```

```
Training Error Rate: 0.036
Training Accuracy: 0.964
Test Error Rate: 0.079
Test Accuracy: 0.921
```

```
# Function to train and evaluate a model with different hyperparameters
def train_and_evaluate(hidden_units=30, learning_rate=0.005, activation=
    """
    Train a neural network with specified hyperparameters

    Args:
        hidden_units: number of neurons in hidden layer
        learning_rate: learning rate for optimizer
        activation: activation function ('relu', 'sigmoid', 'tanh')
        num_epochs: number of times to iterate through training data

    Returns:
        train_accuracy, test_accuracy
    """
    torch.manual_seed(1)  # for reproducibility

    # Define the network
    class Pigeon(nn.Module):
        def __init__(self):
            super(Pigeon, self).__init__()
            self.layer1 = nn.Linear(28 * 28, hidden_units)
            self.layer2 = nn.Linear(hidden_units, 1)
```

```
        def forward(self, img):
            flattened = img.view(-1, 28 * 28)
            activation1 = self.layer1(flattened)

            # Apply specified activation function
            if activation == 'relu':
                activation1 = F.relu(activation1)
            elif activation == 'sigmoid':
                activation1 = torch.sigmoid(activation1)
            elif activation == 'tanh':
                activation1 = torch.tanh(activation1)

            activation2 = self.layer2(activation1)
            return activation2

    pigeon = Pigeon()

    # Training
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.SGD(pigeon.parameters(), lr=learning_rate, momentu

    for epoch in range(num_epochs):
        for (image, label) in mnist_train:
            actual = torch.tensor(label < 3).reshape([1,1]).type(torch.F
            out = pigeon(img_to_tensor(image))
            loss = criterion(out, actual)
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()

    # Evaluate on training set
    error = 0
    for (image, label) in mnist_train:
        prob = torch.sigmoid(pigeon(img_to_tensor(image)))
        if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
            error += 1
    train_accuracy = 1 - error/len(mnist_train)

    # Evaluate on test set
    error = 0
    for (image, label) in mnist_val:
        prob = torch.sigmoid(pigeon(img_to_tensor(image)))
        if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
            error += 1
    test_accuracy = 1 - error/len(mnist_val)

    return train_accuracy, test_accuracy

print("Function defined successfully!")
```

```
Function defined successfully!
```

```python
# Experiment 1: Varying Hidden Units
print("Experiment 1: Varying Hidden Units")
print("="*60)

results_hidden = []
for hidden_units in [10, 30, 100]:
    train_acc, test_acc = train_and_evaluate(hidden_units=hidden_units)
    results_hidden.append({
        'Hidden Units': hidden_units,
        'Train Accuracy': f"{train_acc:.4f}",
        'Test Accuracy': f"{test_acc:.4f}"
    })
    print(f"Hidden Units: {hidden_units:3d} | Train Acc: {train_acc:.4f}

print("\n")
```

```
Experiment 1: Varying Hidden Units
============================================================
Hidden Units:  10 | Train Acc: 0.9530 | Test Acc: 0.8960
Hidden Units:  30 | Train Acc: 0.9640 | Test Acc: 0.9210
Hidden Units: 100 | Train Acc: 0.9700 | Test Acc: 0.9230
```

```python
# Experiment 2: Varying Learning Rate
print("Experiment 2: Varying Learning Rate")
print("="*60)

results_lr = []
for lr in [0.001, 0.005, 0.01]:
    train_acc, test_acc = train_and_evaluate(learning_rate=lr)
    results_lr.append({
        'Learning Rate': lr,
        'Train Accuracy': f"{train_acc:.4f}",
        'Test Accuracy': f"{test_acc:.4f}"
    })
    print(f"Learning Rate: {lr:.3f} | Train Acc: {train_acc:.4f} | Test 

print("\n")
```

```
Experiment 2: Varying Learning Rate
============================================================
Learning Rate: 0.001 | Train Acc: 0.9220 | Test Acc: 0.8870
Learning Rate: 0.005 | Train Acc: 0.9640 | Test Acc: 0.9210
Learning Rate: 0.010 | Train Acc: 0.9610 | Test Acc: 0.9180
```

```python
# Experiment 3: Varying Activation Functions
print("Experiment 3: Varying Activation Functions")
```

```
print("="*60)

results_activation = []
for act in ['relu', 'sigmoid', 'tanh']:
    train_acc, test_acc = train_and_evaluate(activation=act)
    results_activation.append({
        'Activation': act,
        'Train Accuracy': f"{train_acc:.4f}",
        'Test Accuracy': f"{test_acc:.4f}"
    })
    print(f"Activation: {act:7s} | Train Acc: {train_acc:.4f} | Test Acc

print("\n")
```

```
Experiment 3: Varying Activation Functions
============================================================
Activation: relu    | Train Acc: 0.9640 | Test Acc: 0.9210
Activation: sigmoid | Train Acc: 0.9270 | Test Acc: 0.8830
Activation: tanh    | Train Acc: 0.9600 | Test Acc: 0.9060
```

Part (a) -- 3 pt

Comment on which of the above changes resulted in the best accuracy on training data? What accuracy were you able to achieve?

Part (a) Answer:

The configuration that resulted in the best training accuracy was: 100 hidden units achieved a training accuracy of 0.9700

Part (b) -- 3 pt

Comment on which of the above changes resulted in the best accuracy on testing data? What accuracy were you able to achieve?

Part (b) Answer:

The configuration that resulted in the best test accuracy was: Learning rate of 0.005 achieved a test accuracy of 0.9210

Part (c) -- 4 pt

Which model hyperparameters should you use, the ones from (a) or (b)?

Part (c) Answer:

I should use the hyperparameters from part (b) - learning rate of 0.005.

Reasoning:

- Test accuracy measures generalization to unseen data, which is what we care about in practice
- Training accuracy can be misleading if the model is overfitting
- If there's a large gap between training and test accuracy, it indicates overfitting
- The model that performs best on test data is most likely to perform well on new, real-world data

For the combination of hyperparameters, I would use: learning rate = 0.005, hidden units = 100, activation = relu for the final model.